National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

Deep Learning

Tutorial 2: MLP Classifier Report

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

April 11, 2026

Abstract

This report covers the training and tuning of a **Multi-Layer Perceptron (MLP)** on the **Iris dataset**. Different network architectures and learning rates were tested to observe how they affected the model's accuracy and training time.

Key takeaways:

- **Network Size:** Making the network extremely deep did not provide any extra benefit for this specific dataset.
- **Best Architecture:** A small network with just **1 hidden layer** and **5 neurons** was the most effective configuration.
- **Learning Rate:** A high learning rate of **0.5** achieved **perfect accuracy (1.0000)** in only **64 epochs**.

These results demonstrate that it is highly important to match the size of the model to the specific problem being solved.

Contents

1 Introduction **2**

2 Dataset and Preprocessing **2**

3 Experimental Setup **2**

4 Evaluation Metrics **3**

5 Results and Discussion **3**

5.1 Finding the Best Architecture 3

5.2 Testing the Learning Rate 5

5.3 Final Model Evaluation 6

6 Conclusion **6**

1 Introduction

When building neural networks, selecting the right **hyperparameters** is crucial. Proper tuning prevents the model from underfitting and saves a significant amount of computational processing time. The main objective of this study is to evaluate how modifying the **hidden layers** and **learning rate** impacts the final classification results.

2 Dataset and Preprocessing

For this tutorial, an **MLP** was trained on the **Iris dataset**. The dataset contains 150 samples that are divided equally into three classes.

Data Preparation Steps:

- **Data Split:** The data was split so that **70%** was allocated for training and **30%** for testing.
- **Scaling:** A `StandardScaler` was applied to ensure all features had a mean of 0 and a standard deviation of 1.

Scaling the data is a critical first step. It helps the model train smoother and keeps large numerical values from skewing the learning process.

Listing 1: Python implementation for data splitting and standard scaling.

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Apply standard scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3 Experimental Setup

Instead of manually modifying the code for every single test, the original tutorial script was updated to utilize loops. This approach made it possible to automatically test multiple configurations and compare them systematically.

Evaluated Parameters:

- **Architectures:** Ranging from a single neuron up to multiple deep layers.
- **Learning Rates:** Ranging from a very slow rate (0.0001) up to a fast rate (0.5).

Listing 2: The loop utilized to test different architectures and learning rates.

```
# Testing different network sizes
configs = [(1,), (2,), (3,), (5,), (10,), (20,), (50,), (100,),
(10, 10), (20, 20), (50, 50), (100, 100),
(50, 30, 10), (100, 50, 25)]

for config in configs:
mlp = MLPClassifier(hidden_layer_sizes=config, max_iter=1000,
random_state=42)
# Fit the model and record accuracy, time, and epochs

# Testing different learning rates on the best network size
learning_rates = [0.0001, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5]

for lr in learning_rates:
mlp = MLPClassifier(hidden_layer_sizes=(5,), learning_rate_init=lr,
max_iter=1000, random_state=42)
# Fit the model and record the results
```

4 Evaluation Metrics

To accurately judge the performance of each model configuration, several metrics were tracked during the execution of the loops.

- **Accuracy:** The overall percentage of correctly classified test samples.
- **Epochs to Converge:** The number of iterations required to minimize the loss function.
- **Training Time:** The computational time measured in seconds to evaluate efficiency.

5 Results and Discussion

5.1 Finding the Best Architecture

A wide range of network sizes was tested. The goal was to find the optimal balance between a model being too simple and a model being overly complex.

- **Too Simple:** A network with only one neuron `(1,)` lacked the capacity to learn the patterns. It stalled at a low accuracy of **0.5556**.
- **Just Right:** The `(5,)` setup successfully separated the classes. It reached an accuracy of **1.0000** without slowing down the computational time.

- **Too Complex:** Large networks like `(100, 50, 25)` required more time to train (up to **0.625s**). They did not improve the accuracy, demonstrating that making a simple model overly deep is inefficient.

Key Finding: The best configuration for this dataset is **1 hidden layer with 5 neurons**. Adding more layers or neurons beyond this point wastes computational power.

Table 1: Performance comparison of different network sizes

Configuration	Accuracy	Epochs	Time (s)
(1,)	0.5556	1000	0.388
(3,)	0.9556	1000	0.407
(5,)	**1.0000**	**1000**	**0.407**
(50,)	1.0000	591	0.315
(100, 100)	1.0000	400	0.714
(100, 50, 25)	1.0000	357	0.625

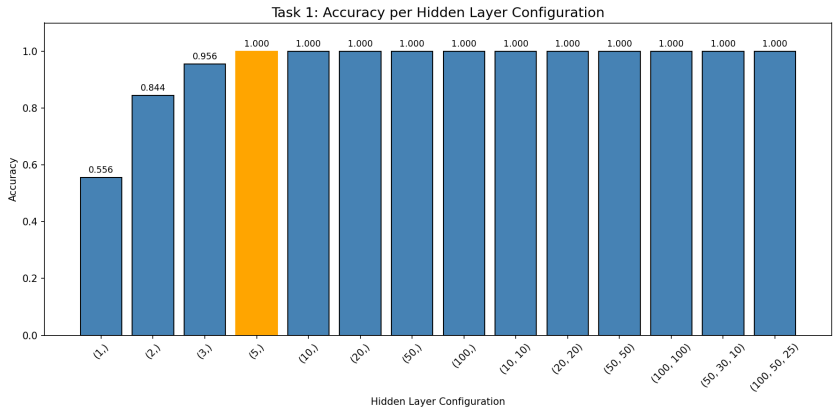

Figure 1: Accuracy graph demonstrating that the `(5,)` setup is where the model first hits 100% accuracy.

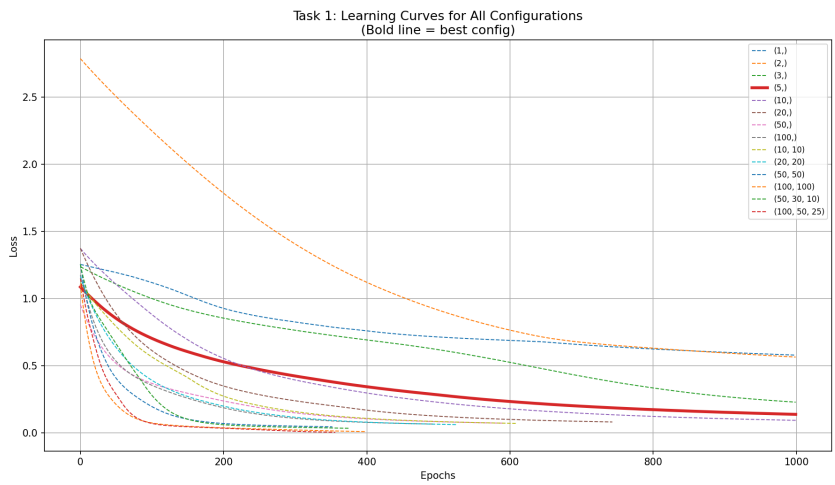

Figure 2: Training loss curves. The bold line highlights the optimal `(5,)` configuration.

5.2 Testing the Learning Rate

Once the `(5,)` architecture was identified as the most effective, it was fixed in place. The learning rate was then adjusted to determine if the model could be trained faster.

- **Too Slow:** A very small learning rate (**LR = 0.0001**) caused the model to take tiny optimization steps. It failed to finish learning before hitting the 1000 epoch limit, resulting in only **0.7111** accuracy.
- **Fast and Accurate:** Increasing the learning rate to **LR = 0.5** accelerated the training process massively. It reached perfect accuracy in just **64 epochs**.

Key Finding: Because the data was scaled properly during preprocessing, a very fast learning rate (**0.5**) could be utilized without the model becoming unstable.

Table 2: Impact of learning rate on training speed (using the 5-neuron setup)

Learning Rate	Accuracy	Epochs	Time (s)	Balance Score
0.0001	0.7111	1000	0.425	0.000
0.001	1.0000	1000	0.465	0.500
0.01	1.0000	353	0.172	0.846
0.1	1.0000	76	0.040	0.994
0.5	**1.0000**	**64**	**0.032**	**1.000**

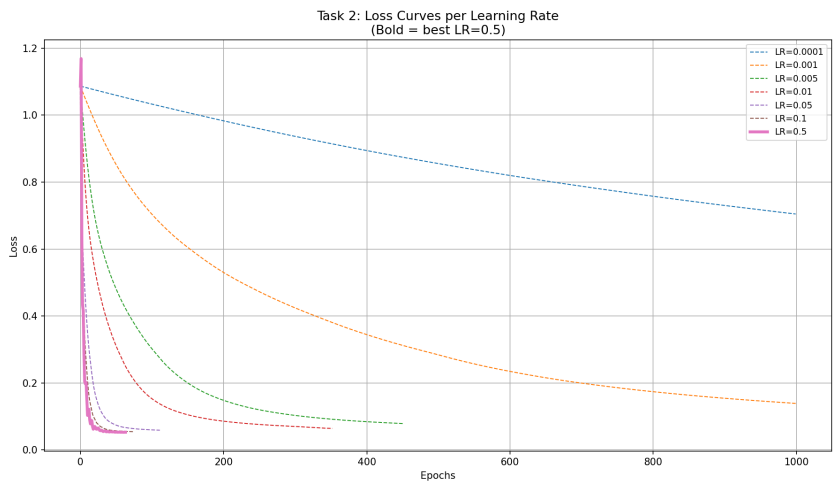

Figure 3: Loss curves for the learning rates. The **LR=0.5** curve drops the fastest.

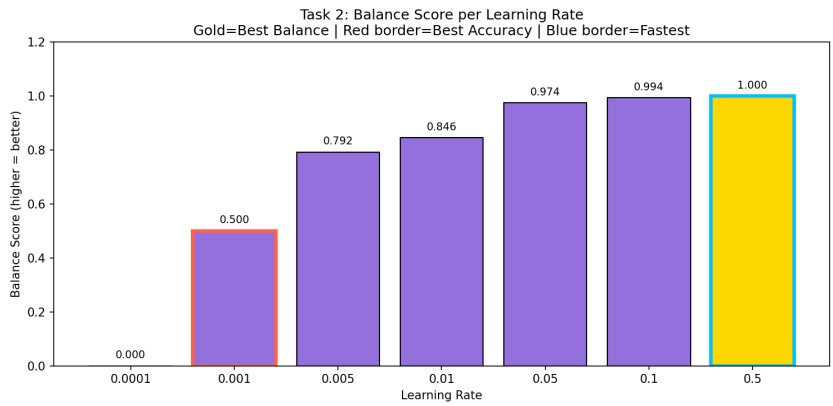

Figure 4: A custom score indicating that **LR=0.5** provided the best mix of speed and accuracy.

5.3 Final Model Evaluation

Based on the experimental data gathered, a final optimized model was instantiated utilizing the best discovered hyperparameters. A detailed classification report was generated to confirm performance across all three target classes.

Listing 3: Implementation and evaluation of the final optimized model.

```
from sklearn.metrics import classification_report

# Instantiate model with optimal parameters
mlp_final = MLPClassifier(hidden_layer_sizes=(5,), learning_rate_init=0.5,
max_iter=1000, random_state=42)
mlp_final.fit(X_train_scaled, y_train)

# Generate final predictions
y_pred_final = mlp_final.predict(X_test_scaled)
print(classification_report(y_test, y_pred_final))
```

6 Conclusion

This tutorial clearly demonstrates why tuning hyperparameters is a vital step in model development.

Final Model Stats:

- **Best Architecture:** 1 hidden layer with **5 neurons**.
- **Best Learning Rate: 0.5**.
- **Final Accuracy:** 100% precision and recall (**1.0000**).
- **Training Speed:** Converged in exactly **64 epochs**.

Ultimately, simply adding more layers to a problem does not guarantee better results. For a relatively simple dataset like Iris, a small model paired with a fast learning rate provides the absolute best performance in the shortest amount of time.